

Step 2：最终 Code Review——技术栈全景回顾

不涉及新代码，这一步是把 Day 1-13 学到的全部内容，系统性地梳理成一张"技术地图"，帮你在脑子里把碎片知识串成一条完整的线。

2.1 技术栈全景表

类别	具体技术/概念	学习日期	在项目里的实际应用
HTML	语义化标签（header/nav/main/section/article/aside/footer）	Day 1	整站的页面骨架
	表单元素（input/textarea/label）	Day 1	联系表单
	<code><meta></code> SEO 标签	Day 13	title + description
CSS	选择器（标签/类/ID/后代/属性）	Day 2	全站样式
	盒模型（margin/padding/border）	Day 2	间距控制
	Flexbox（一维布局）	Day 3	导航栏、Hero 居中
	Grid（二维布局）	Day 9	项目卡片网格
	响应式设计（ <code>@media</code> ）	Day 4	多端适配
JavaScript 基础	<code>:focus-visible</code> 无障碍	Day 13	键盘导航焦点提示
	变量、运算符、条件、循环	Day 5	基础语法能力
	函数（声明式 + 箭头函数）	Day 6	逻辑封装
	数组与数组方法（push/pop/map/filter/reduce）	Day 6、8	数据处理
JavaScript DOM	<code>querySelector</code> / <code>addEventListener</code>	Day 7	导航滚动、表单提交
	<code>classList.toggle()</code>	Day 7	深色模式切换
	数据驱动渲染（ <code>map</code> + <code>innerHTML</code> ）	Day 8	项目卡片动态生成
JavaScript 异步	回调函数、回调地狱	Day 10	理解异步演进历史
	Promise（resolve/reject）	Day 10	异步结果处理
	<code>fetch</code> + <code>async/await</code>	Day 11	真实网络请求
	<code>try/catch</code> 错误处理	Day 11	请求失败处理
性能优化	图片懒加载（ <code>loading="lazy"</code> ）	Day 12	项目卡片图片
	Lighthouse 跑分与优化	Day 12	LCP 从 2.0s 降到 1.0s
Git 本地	init / add / commit / branch / merge	Day 1	版本控制基础
Git 远程	remote add / push / pull / tag / clone	Day 9-13	云端协作与备份
工程素养	代码走查、无障碍、SEO、跨浏览器检查	Day 13	从"能跑"到"可交付"

2.2 代码结构解析（这里换成"知识依赖关系解析"）

```
text

第 1 阶段技术依赖链条
|
HTML（骨架）
├── CSS（外观：Flexbox/Grid/响应式）
│   └── JavaScript 基础语法（变量/函数/数组）
│       ├── DOM 操作（让页面能响应交互）
│           ├── 异步编程（能和外部世界交换数据）
│               ├── 性能优化 + 工程收尾（从能用到好用、可信赖）
│                   └── Git 全流程（贯穿始终，记录每一步成长）
```

关键节点说明： 这条链条不是巧合的教学顺序，而是**真实的技术依赖关系**——没有 HTML 骨架，CSS 无从谈起；没有基础语法，DOM 操作写不出来；没有 DOM 操作的基础，异步编程里"拿到数据后要更新页面"这一步也无从下手。回顾这条链条，能帮你理解"为什么是这个学习顺序"，而不只是记住了一堆孤立的知识点。

2.3 完成标准

不用现在就能对着表格一字不差地背出来，但试着闭上眼睛，凭记忆说出：

- 1. HTML/CSS/JS/Git 四大类里，各自最有把握的 2-3 个知识点
- 2. 整个项目里，你觉得写得最"有成就感"的一段代码是哪个

完成后请回复：

- 你的答案（简短说说就好）
- "回顾完了，进入下一步"